

Computer Vision — RWTH Aachen — Exam Cheat Sheet

Prof. Dr. Bastian Leibe, Visual Computing Institute • 90 min, 6 questions, ≈95–100 marks, **no aids allowed** • topics ordered by past-exam ROI

Q1 Filtering & Patterns conv, Gaussian, Canny, Hough	Q2 Local Features Harris, Hessian, SIFT, <i>H</i>	Q3 Segmentation <i>k</i> -means, mean-shift, MRF, RANSAC	Q4 Sliding-Window Detection HOG, SVM, AdaBoost, Viola–Jones	Q5 CNNs sizes, params, losses, enc-dec	Q6 3D Geometry / Reconstruction <i>K</i> , epipolar, <i>F/E</i> , triangulation, SfM
---	--	---	--	---	---

! marks a recurring examiner trap. Every exam so far has used this same 6-question skeleton.

1 Filtering, Convolution & Fourier

Definitions

Cross-correlation $G = H \otimes F$: $G[i, j] = \sum_{u,v} H[u, v] F[i + u, j + v]$

Convolution $G = H * F$: $G[i, j] = \sum_{u,v} H[u, v] F[i - u, j - v]$

⇒ convolution = **flip the kernel in both dimensions**, then correlate:

$F \otimes H[i, j] = F * H[-i, -j]$.

! *Correlation = convolution* only if $H[-u, -v] = H[u, v]$ (*symmetric kernel*).

Properties (both are linear & shift-invariant)

- Linear: $(aH + J) \otimes F = a(H \otimes F) + (J \otimes F)$ — **always true**.
- Shift-invariant: same operator everywhere.
- Convolution only: commutative, associative, has identity δ , differentiable.
- ! *Correlation is not commutative/associative*.

Boundary handling

Problem: filter support leaves the image ⇒ undefined pixels, dark/bright halo, output shrinks.

- Zero pad**: simple; — dark border, artificial strong edge.
- Wrap (circular)**: consistent with DFT; — mixes opposite sides, nonsense content.
- Clamp / replicate edge**: no new values, cheap; — streaking, biases towards border pixel.
- Mirror / reflect**: smoothest continuation, best in practice; — invents symmetry that isn't there.
- Or: **valid** output only (crop) — no artefacts, but image shrinks.

Separability (marks-rich arithmetic)

2D filter separable $\iff$ rank 1 $\iff H = \mathbf{c} \mathbf{r}^\top$. Apply as row pass then column pass.

Sobel-x: $\begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$ (smooth in y, derive in x).

Cost per pixel, kernel $n \times n$: 2D = n^2 mult + $(n^2 - 1)$ add; separable = $2n$ mult + $2(n - 1)$ add.

$n = 3$: **9 mult + 8 add** vs **6 mult + 4 add**. Box and Gaussian are both separable.

Gaussian smoothing

$G_\sigma(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$, rotationally symmetric, separable, low-pass. Kernel half-width $\approx 3\sigma$ (⇒ size $\approx 6\sigma + 1$).

$G_{\sigma_1} * G_{\sigma_2} = G_{\sqrt{\sigma_1^2 + \sigma_2^2}}$ (two smoothings = one bigger one).

- FT of a Gaussian is a Gaussian (**true**). FT of a box is a **sinc** (not a box).
- Ideal (box) low-pass in frequency ⇒ infinite sinc support in space ⇒ **ringing**. Box filter in space likewise rings; Gaussian does not.
- Duality: better localised in one domain ⇒ worse in the other.
- Truncating the spectrum removes high frequencies ⇒ *blur/ringing*, **! not aliasing**. *Aliasing comes from subsampling, not from low-passing*.

Noise & the median filter

Gaussian noise: $f = \tilde{f} + \eta$, $\eta \sim \mathcal{N}(\mu, \sigma)$, i.i.d. — removed by averaging.

Salt & pepper: random black *and* white pixels. **Impulse**: random white pixels.

Median filter: non-linear, output is an existing pixel value, kills impulse noise, **preserves edges**. Gaussian would smear the outlier over the neighbourhood instead.

Sampling & pyramids

Nyquist: sample at $\geq 2f_{\max}$, else **aliasing**. Always **smooth before subsampling**; lost high frequencies cannot be recovered afterwards.

Gaussian pyramid: $G_0 = I$, $G_i = (G_{i-1} * g) \downarrow 2$. Without the Gaussian: aliasing / moiré, structures appear that are not in the image.

Laplacian pyramid: $L_i = G_i - \text{expand}(G_{i+1})$, top $L_n = G_n$; band-pass, approximates DoG. Reconstruct exactly: $G_i = L_i + \text{expand}(G_{i+1})$ from the top down. Storage $\approx \frac{4}{3}$ of the image, used for compression/blending.

Gradients & edges

$\nabla f = [f_x, f_y]$, magnitude $\|\nabla f\| = \sqrt{f_x^2 + f_y^2}$, direction $\theta = \arctan(f_y/f_x)$ (points across the edge, towards increasing intensity).

Derivative theorem: $\frac{\partial}{\partial x}(h * f) = (\frac{\partial h}{\partial x}) * f$ — differentiate the *filter*, one pass, noise-robust.

LoG: $\nabla^2 G_\sigma$, edges at **zero crossings**; DoG $\approx$ LoG.

Canny edge detector (state all 4–5 steps)

- Smooth with a Gaussian (choose σ : large = robust/coarse, small = fine detail).
- Compute gradient magnitude and orientation (derivative-of-Gaussian filters).
- Non-maximum suppression**: thin to 1-pixel ridges — keep q iff $\text{Mag}(q) > \text{Mag}(p)$ and $\text{Mag}(q) > \text{Mag}(r)$, where p, r are *interpolated* neighbours along the gradient direction.
- Hysteresis thresholding** with $k_{\text{high}}, k_{\text{low}}$ (typ. ratio 2:1): a chain *starts* only above k_{high} , then *continues* through connected pixels above k_{low} .
- Link edges into contours.

Why hysteresis > single threshold: one threshold either fragments strong edges (too high) or admits noise (too low); two thresholds keep weak *connected* continuations while rejecting isolated weak responses. Model assumption: **step edge + additive Gaussian noise**; first derivative of a Gaussian is near-optimal for the detection/localisation trade-off (Canny 1986).

! *Salt & pepper before Canny: each impulse is a huge gradient ⇒ spurious edges + Gaussian smoothing spreads it. Fix: apply a **median filter first**, then Canny.*

Sharpening & templates

Sharpen = $\delta \cdot 2.0$ – (box filter, weight 0.33): accentuates the deviation from the local average. Unsharp mask: $I + \alpha(I - I * G)$.

Filters as templates: filtering = dot product of image patch with the filter vector; “filters look like the effects they find”. Normalise (NCC) or the response just tracks local brightness.

Hough transform (line detection)

Use **polar** parametrisation (slope-intercept $y = mx + c$ blows up for vertical lines):

$$\rho = x \cos \theta + y \sin \theta, \quad \theta \in [0, \pi), \rho \in [-D, D]$$

- Edge detection (e.g. Canny) on $I \Rightarrow$ edge pixels (x, y) (+ gradient orientation).
- Quantise (θ, ρ) into an accumulator array A , init to 0.
- For each edge pixel, for each θ : compute $\rho = x \cos \theta + y \sin \theta$, then $A[\theta, \rho]++$ (a point votes for a *sinusoid* in Hough space).
- Find local maxima of A above a threshold; each is one line.

5. Map (θ, ρ) back to image space; optionally trim to the extent of supporting pixels.

Speed-up: use the gradient orientation at each edge pixel to vote for a single θ instead of all.

Horizontal lines: constant $y \Rightarrow \theta = 90^\circ$ ($\rho = y$) — only accumulate that column, i.e. vote $A[90^\circ, y]++$ and pick its maxima.

Pros: robust to occlusion/gaps, handles multiple lines. Cons: quantisation trade-off, noise gives spurious peaks, cost explodes with parameter count.

2 Local Features

The 5-step pipeline (reproduce in order)

1. **Detect** distinctive, repeatable keypoints.
2. **Define** a region around each keypoint (scale / affine shape).
3. **Extract & normalise** the region content (rotation, illumination).
4. **Compute** a local descriptor from the normalised region.
5. **Match** descriptors: accept if $d(f_A, f_B) < T$ (better: ratio test $d_1/d_2 < 0.8$).

Requirements: **repeatability** (found again under transformation), **distinctiveness**, locality (robust to occlusion/clutter), quantity, efficiency.

Harris corner detector

Auto-correlation / second-moment matrix, g a Gaussian window:

$$M(\sigma_I, \sigma_D) = g(\sigma_I) * \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} (\sigma_D)$$

$$R = \det(M) - \alpha \operatorname{tr}(M)^2 = \lambda_1 \lambda_2 - \alpha (\lambda_1 + \lambda_2)^2, \quad \alpha \approx 0.04-0.06$$

Eigenvalue reading (asked verbatim): both small $\Rightarrow$ **flat** region ($R \approx 0$); one large, one small $\Rightarrow$ **edge** ($R < 0$); both large $\Rightarrow$ **corner** ($R \gg 0$). λ 's are the axes of the error-surface ellipse.

Steps: (1) derivatives I_x, I_y at scale σ_D ; (2) products $I_x^2, I_y^2, I_x I_y$; (3) Gaussian-window sum at σ_I ; (4) cornerness R ; (5) threshold + non-maximum suppression.

Invariance: **translation** yes; **rotation** yes (ellipse rotates, eigenvalues unchanged); intensity *shift* $I + b$ yes, intensity *scaling* aI only up to the threshold; **scale NO**; **affine NO** (only Euclidean/similarity up to threshold; the affine-adapted Harris–Laplace is the extension).

! “Rotation-invariant but not scale-invariant” — a corner at one scale becomes an edge at a larger scale. This is exactly the gap SIFT closes.

Hessian detector

$$\mathcal{H} = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix}, \quad \text{score} = \det(\mathcal{H}) = I_{xx} I_{yy} - I_{xy}^2$$

Differences vs Harris: **second** derivatives of the image vs **first**-derivative products; no windowing/averaging needed; responds to blobs and saddle points rather than to gradient-corner structure; cheaper but noisier (2nd derivatives amplify noise) $\Rightarrow$ smooth first.

Translation-invariant **yes**, rotation-invariant **yes** (det is), scale-invariant **no**.

Typical missing steps in code-completion tasks: (i) use the *smoothed* image's derivatives / apply the Gaussian at the right scale, (ii) scale-normalise the response ($\sigma^4 \det \mathcal{H}$), (iii) **non-maximum suppression** before thresholding.

Automatic scale selection

Evaluate a **scale-normalised** response ($\sigma^2 \nabla^2 G$ for LoG, $\sigma^4 \det \mathcal{H}$) over a range of σ at each location and take the **local maximum over scale** — the characteristic scale covaries with image scale, so the same physical structure is picked in both images. Harris–Laplace = Harris in space $\times$ LoG maximum in scale. DoG (SIFT) approximates LoG cheaply from a Gaussian pyramid.

SIFT (Lowe 2004) — the 128

Region $\rightarrow 4 \times 4$ spatial sub-patches $\times 8$ gradient-orientation bins = $4 \cdot 4 \cdot 8 = 128$ -D.

Rotation invariance: rotate the patch to the **dominant gradient orientation** (peak of the orientation histogram). Illumination: normalise, clip at 0.2, renormalise (affine intensity change).

Robust to $\sim 60^\circ$ out-of-plane rotation and large illumination change. **SURF:** box filters + integral images, $\sim 6\times$ faster at similar quality.

Raw pixel patches as descriptors: only work for pure translation with identical illumination — no invariance to rotation/scale/lighting, high-dimensional and noise-sensitive.

Homography estimation (DLT)

$p_r = H p_l$ in homogeneous coordinates, equality **up to scale** $\Rightarrow$ take the cross product $p_r \times H p_l = 0$; two independent rows per correspondence:

$$\begin{bmatrix} 0 & 0 & 0 & -x_l & -y_l & -1 & y_r x_l & y_r y_l & y_r \\ x_l & y_l & 1 & 0 & 0 & 0 & -x_r x_l & -x_r y_l & -x_r \end{bmatrix} \mathbf{h} = 0$$

Derivation: with $\mathbf{h} = (h_{11} \dots h_{33})^\top$, $x_r = \frac{h_{11}x_l + h_{12}y_l + h_{13}}{h_{31}x_l + h_{32}y_l + h_{33}}$ and $y_r = \frac{h_{21}x_l + h_{22}y_l + h_{23}}{h_{31}x_l + h_{32}y_l + h_{33}}$; cross-multiply and collect terms.

DoF: H has 9 entries, defined up to scale $\Rightarrow$ **3 DoF**; each correspondence gives 2 equations $\Rightarrow$ **4 point correspondences** (no 3 collinear). Solve $A\mathbf{h} = 0$ by SVD: $\mathbf{h}$ = right singular vector of the **smallest** singular value ($\|\mathbf{h}\| = 1$ to avoid the trivial solution). Normalise coordinates first for conditioning.

RANSAC

For line fitting:

1. Randomly sample the **minimal** set (2 points for a line, 4 for H , 3 for a plane, 8 for F).
2. Fit the model to that sample.
3. Count **inliers**: points with residual $<$ threshold t .
4. Keep the model with the most inliers.
5. Repeat k times; finally **re-fit** on all inliers of the best model (least squares).

Iteration count. Let δ be the outlier ratio, s the sample size, ε the allowed failure probability.

$P(\text{one sample all inliers}) = (1 - \delta)^s$; $P(\text{that sample fails}) = 1 - (1 - \delta)^s$;

$$P(\text{fail}) = (1 - (1 - \delta)^s)^k \leq \varepsilon \Rightarrow k \geq \frac{\log \varepsilon}{\log (1 - (1 - \delta)^s)}$$

(the log of a number < 1 is negative, so **the inequality flips** on division). For H , $s = 4$.

3D plane version: sample 3 points, model $n^\top X + d = 0$ (normal from the cross product of two edge vectors), residual = point-to-plane distance $|n^\top X + d|/\|n\|$; everything else identical.

Pros: simple, general, tolerates $> 50\%$ outliers. Cons: k grows fast with s and δ ; needs a threshold; not deterministic.

3 Segmentation & Clustering

k-means (steps)

1. Choose k and initialise the cluster centres $\mu_1 \dots \mu_k$ (e.g. random points).
2. **Assign** each point to the nearest centre: $\gamma_n = \arg \min_j \|x_n - \mu_j\|^2$.
3. **Update** each centre to the mean of its assigned points.
4. Repeat 2–3 until assignments (or centres) no longer change.

Objective $J = \sum_n \sum_j \gamma_{nj} \|x_n - \mu_j\|^2$; both steps decrease $J \Rightarrow$ **always converges (Yes)**; converges to a **local** optimum only (**No, not the best solution**); finding the global optimum is **NP-hard (Yes)**.

+ Simple, fast ($O(nkd)$ per iteration), guaranteed to converge.

– Must choose k ; sensitive to initialisation and outliers; assumes isotropic, equally-sized, convex (spherical) clusters — fails on elongated/non-convex data; hard assignments.

For segmentation: represent each pixel as a feature vector (intensity, or RGB, or RGB + (x, y) for spatial coherence, or texture/filter responses), cluster with k -means, then label every pixel with its cluster index; connected components of one label form segments. Adding (x, y) (scaled) trades colour purity for spatial compactness.

Mean-shift (steps)

1. Choose a kernel and **bandwidth** r ; place a search window at a data point.
2. Compute the **mean** (centre of mass) of the points inside the window.
3. **Shift** the window centre to that mean.
4. Repeat 2–3 until convergence (a mode of the density).
5. Repeat for every data point; points converging to the same mode form one cluster (merge modes within r).

+ Non-parametric — no assumption on cluster shape; **number of clusters is found automatically**; single robust parameter; finds density modes, robust to outliers.

– Must choose the bandwidth r (result is very sensitive to it); computationally expensive ($O(n^2)$ per iteration); scales badly with feature

dimension; may not work well with a fixed bandwidth on varying densities.

Speed-up: assign *all* points along a convergence trajectory to the same mode (“basin of attraction”); also assign points within r of the path; subsample/use a grid or approximate nearest-neighbour lookups.

Segmentation: cluster pixels in joint colour+position space (r, g, b, x, y) with separate colour/spatial bandwidths (h_r, h_s) ; each mode is a segment.

r larger $\Rightarrow$ fewer, coarser clusters; r smaller $\Rightarrow$ more, finer clusters.

Mixture of Gaussians & EM

$p(x|\theta) = \sum_j \pi_j \mathcal{N}(x|\mu_j, \Sigma_j)$, $\sum_j \pi_j = 1$. Soft version of k -means.

E-step (responsibilities): $\gamma_j(x_n) = \frac{\pi_j \mathcal{N}(x_n|\mu_j, \Sigma_j)}{\sum_k \pi_k \mathcal{N}(x_n|\mu_k, \Sigma_k)}$

M-step: $\hat{N}_j = \sum_n \gamma_j(x_n)$, $\mu_j = \frac{1}{N_j} \sum_n \gamma_j(x_n) x_n$, $\Sigma_j = \frac{1}{N_j} \sum_n \gamma_j(x_n) (x_n - \mu_j)(x_n - \mu_j)^\top$, $\pi_j = \hat{N}_j / N$.

Iterate; the likelihood increases monotonically, converges to a local optimum. k -means = EM with hard assignments and $\Sigma \rightarrow 0$.

Bayes / discriminative vs generative

$p(C_k|x) = \frac{p(x|C_k)p(C_k)}{\sum_j p(x|C_j)p(C_j)}$; decide $\arg \max_k p(C_k|x)$.

Generative: model likelihood $p(x|C_k)$ + prior (MoG, colour histograms). **Discriminative:** model the posterior directly (SVM, CNN).

With a colour histogram likelihood: $p(c|\{r, g, b\}) = \frac{p(\{r, g, b\}|c)p(c)}{\sum_{c'} p(\{r, g, b\}|c')p(c')}$.

MRF energy

$$p(x, y) = \prod_i \Phi(x_i, y_i) \prod_{i, j \in \mathcal{N}} \Psi(x_i, x_j) \\ \iff E(x, y) = \sum_i \phi(x_i, y_i) + \sum_{i, j \in \mathcal{N}} \psi(x_i, x_j)$$

$x_i \neq$ hidden label of pixel i (of g bg or b class), y_i = observation (colour) at ϕ = **unary / data term**: cost of giving pixel i label x_i given its own appearance, $\phi = -\log p(y_i|x_i)$.

ψ = **pairwise / smoothness term**: cost of neighbouring pixels disagreeing — enforces spatial coherence, removes speckle.

Objective: $\hat{x} = \arg \min_x E(x, y)$ (MAP = energy minimisation, since $E = -\log p$ up to a constant).

Potts: $\psi = \theta_\psi \delta(x_i \neq x_j)$ — flat penalty per label change.

Contrast-sensitive Potts: $\psi = \theta_\psi g_{ij}(y) \delta(x_i \neq x_j)$ with $g_{ij}(y) = e^{-\|y_i - y_j\|^2 / B}$ — cheap to cut where the image itself has a strong edge, so boundaries snap to image contours.

Graph cuts / max-flow

Build a graph: one node per pixel; **source** s (= foreground) and **sink** t (= background) terminals.

- Edge $s \rightarrow i$ with capacity = cost of labelling i *background* (i.e. the fg data term / fg potential), edge $i \rightarrow t$ with capacity = cost of labelling i *foreground*. (Be explicit about your convention in the exam.)
- Edge $i - j$ between neighbours with capacity = ψ_{ij} (pairwise).

An $s-t$ cut assigns every node to s (fg) or t (bg); **cut cost** = **energy**. Solve exactly by **min-cut** = **max-flow** (Ford-Fulkerson / BK).

Hard constraints (e.g. “all border pixels must be background”): set the capacity of the s -edge of those pixels to 0 and of their t -edge to ∞ — an infinite edge can never be cut, forcing the label. (Brush strokes in interactive segmentation work the same way.)

Submodularity is required for a global optimum: $E(s, s) + E(t, t) \leq E(s, t) + E(t, s)$ (equivalently, non-negative pairwise costs that favour agreement). Only **binary, submodular** energies are guaranteed globally optimal.

α -expansion: exact multi-label is NP-hard $\Rightarrow$ iterate binary cuts “keep your label or switch to α ” over all α ; gives a **2-approximation** for metric pairwise terms.

GrabCut: user draws a box $\Rightarrow$ initialise fg/bg MoG colour models $\Rightarrow$ graph cut $\Rightarrow$ **re-estimate the MoG parameters (means, covariances, weights π , and the pixel-to-component assignments) from the new masks** $\Rightarrow$ repeat. That is what the iteration updates: the appearance model, not the pairwise weights ($\gamma \approx 25$ is fixed).

! *Why k -means / MoG / MaxFlow are bad plane-fitters: k -means and MoG partition points into compact/elliptical blobs and are pulled by outliers — they have no notion of “fits a geometric model”; MaxFlow needs a graph with a data term, i.e. labels/appearance you do not have. RANSAC is model-fitting with explicit outlier rejection.*

4 Sliding-Window Object Detection

Setup

Classifier answers “what is in this image?” (one label per image); **detector** answers “what is where?” — it must also localise (bounding boxes) and cope with a variable number of instances plus background.

Approach: slide a fixed-size window over every position *and* scale (image pyramid), extract features in each window, score with a binary classifier, threshold, then **non-maximum suppression** over overlapping boxes. Also over aspect ratios/orientations if needed. Brute force: e.g. 250 000 locations $\times$ 30 orientations $\times$ 4 scales = 30 000 000 evaluations — hence cascades and region proposals.

Assumes the object is well-approximated by a rigid box with limited intra-class deformation.

HOG (Dalal & Triggs, CVPR 2005)

Chain: normalise gamma/colour $\rightarrow$ compute gradients $\rightarrow$ per-cell orientation histogram (locally orderless, vote weighted by magnitude) $\rightarrow$ group cells into overlapping **blocks** and contrast-normalise $\rightarrow$ concatenate into one vector $\rightarrow$ linear SVM.

Motivation: gradient *orientation* captures shape/silhouette and is largely invariant to illumination and colour; the cell histogram is invariant to small shifts and deformations — ideal for pedestrians (upright, characteristic contour, varied clothing).

Size arithmetic (window $W \times H$ px, cell $c \times c$, B bins, no block overlap): $\frac{W}{c} \cdot \frac{H}{c} \cdot B$ values. E.g. 200×100 , $c = 10$, $B = 8$: $10 \times 20 \times 8 = 1600$.

Linear SVM parameters: one weight per feature + bias = **1601**.

Training vectors: one per training image = $1000 + 1000 = 2000$ (positives + negatives; hard-negative mining adds more).

! *1000 RGB pedestrian images alone are not enough: a sliding-window SVM needs **negative** examples too (and identically sized, aligned crops). With positives only you cannot learn a decision boundary.*

Linear SVM

Find the hyperplane $w^\top x + b = 0$ separating the classes that **maximises the margin** — the distance to the closest training points. Those closest points are the **support vectors**, satisfying $w^\top x_n + b = \pm 1$; margin = $2/\|w\|$, so we minimise $\frac{1}{2}\|w\|^2$ subject to $t_n(w^\top x_n + b) \geq 1$ ($t_n \in \{-1, +1\}$). Soft margin: allow slack ξ_n with penalty C . Classify: $f(x) = \text{sign}(w^\top x + b)$.

Maximum margin $\Rightarrow$ best expected generalisation; the solution depends only on the support vectors.

Kernel trick: the dual depends on the data only through inner products $\phi(x_i)^\top \phi(x_j)$, so replace them with a kernel $K(x_i, x_j)$ and never compute ϕ explicitly. This buys a non-linear boundary in input space at linear-SVM cost, even for infinite-dimensional ϕ . Kernels: linear $x_i^\top x_j$; polynomial $(1 + x_i^\top x_j)^p$; Gaussian/RBF $\exp(-\|x_i - x_j\|^2 / 2\sigma^2)$.

AdaBoost

Input: labelled training set $\{(x_n, t_n)\}$, a pool/family of weak learners, number of rounds M . **Output:** strong classifier $H(x) = \text{sign}(\sum_{m=1}^M \alpha_m h_m(x))$ — the selected weak classifiers h_m and their weights α_m .

Steps: (1) initialise uniform data weights $w_n = 1/N$; (2) for $m = 1..M$: train/select the weak classifier with the lowest **weighted** error ϵ_m ; (3) compute its weight $\alpha_m = \frac{1}{2} \ln \frac{1-\epsilon_m}{\epsilon_m}$; (4) **re-weight the data:** increase w_n on *misclassified* points, decrease on correct ones, renormalise; (5) next round focuses on the previously hard examples; (6) output the weighted vote.

So after each round you adjust: α_m **up** when ϵ_m is **small**, and the **sample weights** — **up** for **misclassified**, **down** for **correctly classified**.

Requirement on weak learners: only better than chance, $\epsilon_m < 0.5$ (“weak learnability”); they may be barely better, and should be fast/simple.

Viola-Jones face detection

Weak classifiers = **Haar-like features**: sums of pixels in adjacent black/white rectangles (2-, 3-, 4-rectangle patterns) subtracted from each other, thresholded: $h(x) = 1$ if $p f(x) < p \theta$. They encode simple contrast patterns (eye region darker than cheeks, nose bridge brighter than eyes). Huge over-complete pool ($\sim 180k$ in a 24×24 window) $\Rightarrow$ AdaBoost also performs **feature selection**.

Integral image $II(x, y) = \sum_{x' \leq x, y' \leq y} I(x', y')$ — computed in one pass. Any axis-aligned rectangle sum is then $A - B - C + D$: **4 lookups, 3 operations, constant time independent of rectangle size**. That is what makes evaluating hundreds of thousands of Haar features at every

window and scale feasible.

Cascade: chain of increasingly complex boosted classifiers. Each stage is tuned for a very high detection rate ($\approx 99.9\%$) and only a modest false-positive rate ($\approx 50\%$); a window rejected at any stage is discarded immediately, only survivors reach the expensive stages. Since the vast majority of windows are background, average cost collapses. Overall detection rate = $\prod_i d_i$, false positives = $\prod_i f_i$.

! *Roerei / non-rectangular shapes (circles, triangles): the integral image trick no longer applies — constant-time area sums only work for axis-aligned rectangles, so evaluation cost explodes. Potential advantage: shapes better matched to object contours could be more discriminative per feature, so fewer features might be needed.*

Beyond brute force

DPM (Felzenszwalb et al.): coarse root filter + finer part filters + deformation cost; $\text{score}(z) = \sum(\text{filter responses}) - \sum(\text{displacement penalties}) = \beta^\top \Psi(H, z)$; optimised efficiently with the **generalised distance transform**. Handles articulation/deformation that a rigid template cannot.

Region proposals (Selective Search): bottom-up superpixel segmentation, hierarchically merged $\Rightarrow$ a few thousand class-agnostic candidate regions instead of millions of windows.

5 Convolutional Neural Networks

Why convolution (the $10^{12} \rightarrow 10^8 \rightarrow 10^4$ argument)

Fully connected, 1000×1000 image, 10^6 hidden units: 10^{12} parameters. Restrict to **local** 10×10 connectivity: 10^8 . Add **weight sharing** — 100 learned 10×10 filters: 10^4 parameters, output $1000 \times 1000 \times 100$.

So: local receptive fields + shared weights $\Rightarrow$ far fewer parameters, translation **equivariance**, and features that generalise across the image; an MLP would have to relearn the same pattern at every location and discards the spatial structure.

Layer pipeline: convolution (learned) $\rightarrow$ non-linearity $\rightarrow$ spatial pooling $\rightarrow$ normalisation.

Sizes & parameter counts (do the arithmetic, show it)

$$O = \frac{I - F + 2P}{S} + 1 \quad (\text{per spatial dimension})$$

Conv params = $F_H F_W C_{in} C_{out} + C_{out}$ (bias). BatchNorm params = $2C_{out}$ (γ, β). ReLU/pooling: 0.

Worked: RGBD ($C_{in} = 4$) 100×100 , 3×3 , $S = 1$, $P = 1$, $C_{out} = 10$, block = Conv $\rightarrow$ ReLU $\rightarrow$ BN: $3 \cdot 3 \cdot 4 \cdot 10 + 10 = 370$; $+ 2 \cdot 10 = 20 \Rightarrow 390$. Output $100 \times 100 \times 10$.

! *With a 1000×1000 input the answer is still 390 — convolution parameters do not depend on the input resolution. Only the activation size changes.*

Grayscale $\rightarrow 100$ ch., 3×3 , $S=1$, $P=1$ on 32×32 : $3 \cdot 3 \cdot 1 \cdot 100 + 100 = 1000$ params; output values = $32 \cdot 32 \cdot 100 = 102400$.

Receptive field: stacking L conv layers of size F , stride 1 $\Rightarrow$ RF

= $1 + L(F - 1)$. Two $3 \times 3 = 5 \times 5$; four $3 \times 3 = 9 \times 9$ (with fewer parameters and more non-linearities than one 9×9).

Enlarge the RF without more convs: **pooling / striding** (downsample), **dilated (atrous) convolutions**, an encoder-decoder (downsample then upsample), or global/self-attention.

Layer zoo

Pooling (max/avg): no parameters, applied *per channel* independently, downsamples, gives local translation robustness. **Unpooling**: place the value in a fixed position (e.g. top-left) or replicate; **max-unpooling**: put the value back at the **remembered argmax position** from the corresponding max-pool layer, zeros elsewhere — preserves spatial detail, requires the stored switches.

1×1 **conv**: changes channel depth only, $C_{in} C_{out}$ params, = an FC layer applied per pixel; used for cheap dimensionality reduction (Inception, bottlenecks).

Grouped conv: split channels into G groups (C_{in}, C_{out} must be multiples of G), $G \times$ fewer parameters. **Depthwise**: $G = C_{in} = C_{out}$; $+ 1 \times 1$ pointwise = **depthwise separable**.

Transposed conv (learned upsampling): $H_{out} = S(H_{in} - 1) + F - 2P$.

! *If the filter size is not a multiple of the stride $\Rightarrow$ uneven overlap $\Rightarrow$ checkerboard artifacts.*

Non-linearities: $\sigma(a) = \frac{1}{1+e^{-a}}$, $\tanh(a) = 2\sigma(2a) - 1$, ReLU = $\max(0, a)$ (default: no saturation for $a > 0$, fast, sparse).

BatchNorm normalises over the batch (and H, W) per channel; **Layer-Norm** over channels/features per sample (needed for transformers/small batches).

Losses (write the formula and define every symbol)

L_1 : $\mathcal{L} = \frac{1}{n} \sum_{i=1}^n |f(x_i) - t_i|$ (per-pixel/per-channel mean for images) — robust to outliers, sharper image outputs than L_2 .

L_2 : $\mathcal{L} = \frac{1}{n} \sum_i \|f(x_i) - t_i\|_2^2$.

Binary CE: $\mathcal{L} = -\frac{1}{n} \sum_i [t_i \log \hat{y}_i + (1 - t_i) \log(1 - \hat{y}_i)]$.

Multi-class CE: $\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K t_{ik} \log \hat{y}_{ik}$, $\hat{y}_{ik} = \text{softmax}(f_k(x_i))$, t_{ik} the one-hot label.

Hinge (SVM): $\mathcal{L} = \sum_n [1 - t_n f(x_n)]_+$.

n = minibatch size, x_i input, t_i target, $\hat{y}$ prediction after softmax/sigmoid.

Semantic segmentation: cross-entropy **does** apply — it is just per-pixel classification, so average CE over all pixels (optionally class-weighted, or + Dice/IoU loss for class imbalance). For *regression* outputs (colourisation, depth) use L_1/L_2 instead.

Regularisers: L_2 (weight decay) $\Omega(w) = \frac{1}{2} \|w\|_2^2$ shrinks weights; L_1 $\Omega(w) = \frac{1}{2} \|w\|_1$ gives sparsity / feature selection. Also dropout, augmentation, early stopping.

Backprop: forward pass computes the loss; backward pass applies the chain rule over the computational graph (automatic differentiation), reusing intermediates.

Architectures (year, idea, numbers)

LeNet 1998 (2 conv + 2 pool + FC) · **AlexNet** 2012 (7 layers, 60M params, ReLU, dropout, top-5 error 16.4%) · **VGG** (19 layers, $\sim 150M$, only stacked 3×3) · **GoogLeNet** (Inception modules, 1×1 reduction, only 6.8M) · **ResNet** ($F(x) = H(x) + x$ residual/skip, 100+ layers, BatchNorm — skips fix vanishing gradients and degradation) · **EfficientNet** (compound scaling $d = \alpha^\phi, w = \beta^\phi, r = \gamma^\phi, \alpha\beta^2\gamma^2 \approx 2$) · **ConvNeXt** 2022 (depthwise conv, LayerNorm, GeLU — a “modernised” CNN matching ViTs).

Encoder-decoder & segmentation

Encoder downsamples: large receptive field, global/semantic context, cheap deep computation — but loses spatial resolution. Decoder upsamples back to the input resolution so we can predict a label *per pixel*.

Skip connections (U-Net) pass high-resolution, low-level features from encoder to decoder: the decoder alone cannot recover boundary detail destroyed by pooling, and the skips also improve gradient flow. **FPN**: multi-scale feature pyramid for dense/multi-scale prediction.

Ways to downsample in the encoder: **strided convolution** (learned, $S > 1$) and **pooling** (max/average, no parameters); also space-to-depth / 1×1 after striding.

Matching / metric learning

Siamese: two weight-shared copies (W) embed two inputs; compare $\|D(x_1) - D(x_2)\|_2$.

Three task formulations: **Identification** (multi-class classification loss), **Embedding** (continuous space, dot product / large-margin / triplet loss), **Verification** (binary same-different).

Triplet loss: anchor a , positive p , negative n ; want $D_{a,p} < D_{a,n}$; with margin m : $\mathcal{L} = \sum [m + D_{a,p} - D_{a,n}]_+$.

Triplets grow cubically and most become uninformative $\Rightarrow$ **hard mining**.

Offline: embed the whole set, mine, update (wasteful, stale). **Online** (preferred): sample P identities $\times K$ images per batch and form triplets inside the batch. **Batch All** sums all valid triplets; **Batch Hard** takes, per anchor, the hardest positive and hardest negative.

Contrastive learning: pull augmentations of the same sample together, push different samples apart (SimCLR/DINO); **CLIP** learns a joint image-text space with a contrastive loss on the diagonal of the $N \times N$ similarity matrix.

6 3D Geometry & Reconstruction

Camera model

$$\tilde{x} = K[R | t] \tilde{X} = P \tilde{X}, \quad K = \begin{bmatrix} \alpha_x & s & c_x \\ 0 & \alpha_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

$\alpha_x = f m_x$, $\alpha_y = f m_y$: focal length in pixels along x/y (differ if pixels are not square); s : **skew** (non-rectangular pixels, usually 0); (c_x, c_y): **principal point**, where the optical axis pierces the image plane (usually near the image centre). K has **5 DoF** (intrinsic); $[R | t]$ has **6 DoF** (ex-

intrinsics: 3 rotation, 3 translation); P is 3×4 , defined up to scale $\Rightarrow$ **11 DoF**.

Perspective projection: $x = f \frac{X}{Z}$, $y = f \frac{Y}{Z}$ — non-linear in Euclidean coordinates, linear in homogeneous coordinates. Parallel lines meet at vanishing points; length and angle are not preserved.

Camera centre from P

C is the point projecting to nothing: $PC = 0 \Rightarrow C$ is the **null space** (right null vector) of P . With $P = [M \mid p_4]$ (M is 3×3): $\tilde{C} = -M^{-1}p_4$, homogeneous $C = (\tilde{C}, 1)$.

```
def camera_center_from_projection_matrix(P):
    # null space of P via SVD: last row of Vt
    U, S, Vt = np.linalg.svd(P)
    C = Vt[-1]
    return C[:3] / C[3] # dehomogenise
# equivalently: C = -np.linalg.inv(P[:3, :3]) @ P[:3, 3]
```

Epipolar geometry — vocabulary

Baseline: the line joining the two camera centres O, O' . **Epipole** e (e'): the projection of the *other* camera centre into this image = the intersection of the baseline with the image plane; **all epipolar lines pass through it**. **Epipolar plane**: the plane spanned by the two centres and the 3D point X — it contains O, O', X and both rays. **Epipolar line**: the intersection of the epipolar plane with an image plane = the image of the other camera's ray. **Epipolar constraint**: the match for x must lie on the epipolar line $l' = Fx$ — reduces correspondence search from 2D to **1D**.

Essential & fundamental matrix

Derivation (calibrated / normalised coordinates). Let X be the point in the left camera frame and $X' = RX + T$ in the right frame, where (R, T) is the relative pose. In the right frame the three vectors X', T (the baseline) and RX all lie in the **epipolar plane**, and $X' - T = RX$, so they are **coplanar**:

$$X' \cdot (T \times RX) = 0 \implies X'^T [T]_{\times} RX = 0$$

$$X'^T \underbrace{[T]_{\times} R}_E X = 0, \quad [T]_{\times} = \begin{bmatrix} 0 & -T_z & T_y \\ T_z & 0 & -T_x \\ -T_y & T_x & 0 \end{bmatrix}$$

$E = [T]_{\times} R$: relates **normalised** (calibrated) image coordinates; 5 DoF (3 rotation + 2 translation direction, scale-free); $\det E = 0$, two equal singular values.

$F = K'^{-T} E K^{-1}$: same relation for **raw pixel** coordinates, so it needs **no calibration**; $x'^T F x = 0$; 7 DoF (9 entries — scale — rank constraint). Epipolar lines: $l' = Fx$, $l = F^T x'$; epipoles: $Fe = 0$, $F^T e' = 0$.

Eight-point algorithm

Write out $y^T F x = 0$ with $x = (x_1, x_2, 1)$ (left) and $y = (y_1, y_2, 1)$ (right): $\sum_{i,j} y_i F_{ij} x_j = 0$, so the coefficient of F_{ij} is $y_i x_j$. Row (for $\mathbf{f} = (F_{11}, F_{12}, F_{13}, F_{21}, \dots, F_{33})^T$):

$$[y_1 x_1 \ y_1 x_2 \ y_1 \ y_2 x_1 \ y_2 x_2 \ y_2 \ x_1 \ x_2 \ 1]$$

Stack ≥ 8 such rows $\Rightarrow Af = 0$. **Solve by SVD**: $\mathbf{f}$ = the right singular vector belonging to the **smallest singular value** of A (the least-squares solution under $\|\mathbf{f}\| = 1$); F is only defined **up to scale**, hence the norm constraint — otherwise $\mathbf{f} = 0$.

Why it is inaccurate with noise: the entries of A mix terms of wildly different magnitude ($x_1 y_1 \sim 10^5$ vs 1) $\Rightarrow A$ is very badly conditioned, and the algebraic least-squares error being minimised has no geometric meaning.

Fix — Hartley normalisation: translate each image's points so the centroid is at the origin and scale so the mean distance is $\sqrt{2}$ (T, T'); run the 8-point algorithm on the normalised points; then de-normalise $F = T'^T \hat{F} T$. (Optionally refine by minimising the geometric/Sampson error.)

Rank: $\text{rank}(F) = 2$, $\det F = 0$. Reason: $F = [e']_{\times} H$ maps every point to a line through the epipole, and $[e']_{\times}$ (a skew-symmetric 3×3) has rank 2 — the epipole is the null vector, $Fe = 0$.

If F had full rank: no non-trivial null vector $\Rightarrow$ **no epipole**, the computed epipolar lines would not all intersect in one point, so the geometry would be inconsistent and correspondences would be searched along wrong lines.

Enforcing rank 2: SVD the estimate, $\hat{F} = U \text{diag}(\sigma_1, \sigma_2, \sigma_3) V^T$, set $\sigma_3 = 0$, recompose $F = U \text{diag}(\sigma_1, \sigma_2, 0) V^T$ — the closest rank-2 matrix in Frobenius norm.

Triangulation (linear / DLT)

Given $x \cong PX$, the cross product vanishes: $x \times PX = 0$ gives 2 independent equations per view:

$$\begin{bmatrix} x p_3^T - p_1^T \\ y p_3^T - p_2^T \\ x' p_3^T - p_1^T \\ y' p_3^T - p_2^T \end{bmatrix} X = 0 \implies AX = 0$$

($p_i^T = i$ -th row of P .) Solve by SVD (smallest singular vector), de-homogenise. Geometrically: the two rays generally do **not** intersect because of noise, so we take the algebraic least-squares point (better: minimise reprojection error $\Rightarrow$ non-linear, “optimal” triangulation).

Needs: calibration (both P 's) + correspondences. Degenerate when the point lies on the baseline / the rays are nearly parallel (small baseline $\Rightarrow$ large depth uncertainty).

Stereo

Rectified, parallel cameras with baseline B and focal f : $Z = \frac{fB}{d}$, disparity $d = x_l - x_r$. Disparity $\propto 1/Z$: far points barely shift. **Rectification** warps both images so epipolar lines are horizontal and corresponding $\Rightarrow$ 1D scanline search.

Window matching: SSD/SAD/NCC (NCC handles gain/bias changes). Problems: textureless regions, repetitive patterns, occlusions, specularities. Window size trades noise (small) against blurred boundaries (large).

Add ordering/smoothness constraints or a global (graph-cut/MRF) formulation.

Accuracy improves with a larger baseline, but matching gets harder (more foreshortening, more occlusion).

Structure from Motion

Given m views of n points, recover camera poses and 3D structure jointly (bundle adjustment minimises reprojection error).

$$2mn \geq 11m + 3n - 15:$$

$2mn = \text{measurements}$: each of n points in each of m images gives 2 image coordinates.

$11m = \text{unknowns for the cameras}$: each projective camera P is $3 \times 4 = 12$ entries defined up to scale $\Rightarrow$ 11 DoF each.

$3n = \text{unknowns for the structure}$: each 3D point has 3 coordinates. $-15 = \text{the projective ambiguity}$: $P_i H^{-1}$ and $H X_j$ give identical images, and a 4×4 homography H has $16 - 1 = 15$ DoF — so 15 of the unknowns are not recoverable and must be subtracted (gauge freedom). With calibrated cameras the ambiguity shrinks to a similarity (7 DoF: scale + rotation + translation).

Pipeline: detect & match features $\rightarrow$ estimate F/E with RANSAC $\rightarrow$ decompose to $R, t \rightarrow$ triangulate $\rightarrow$ incrementally add views (PnP) $\rightarrow$ bundle adjustment. Metric reconstruction needs calibration; otherwise it is up to a projective transformation, and the absolute scale is never recoverable from images alone.

7 Attention & Transformers

Scaled dot-product attention (4 named steps)

Alignment $E = QK^T \rightarrow$ **Scale** $\hat{E} = E / \sqrt{D_k}$ (reduces the variance, keeps the softmax smooth / gradients alive) $\rightarrow$ **Weights** $A = \text{softmax}(\hat{E})$ (softmax *across keys*) $\rightarrow$ **Weighted sum** $C = AV$.

$Q = XW_q, K = XW_k, V = XW_v$ from the same $X \Rightarrow$ **self-attention**; K, V from a second source $Y \Rightarrow$ **cross-attention** (token counts may differ).

Masked attention: add a (causal, lower-triangular) mask before the softmax so a position cannot attend to the future — autoregressive decoding.

Multi-head: split the feature dimension, $\sum_h d_h = d$, run heads in parallel, concatenate — diverse subspaces at no extra cost.

Complexity $O(N^2)$ in the sequence length (the full pairwise matrix) — the main scaling cost.

! *Stacking self-attention layers with no non-linearity between them just re-averages value vectors — the per-token MLP sublayer is what gives a transformer its expressive power.*

LayerNorm placement: original/post-norm $\text{LN}(x + \text{SubLayer}(x))$; modern/pre-norm (more stable) $\text{SubLayer}(\text{LN}(x)) + x$.

Positional encoding: attention is permutation-invariant, data is not. Wanted: unique per position, consistent relative distances across lengths, generalises to longer sequences, bounded, deterministic. Sinusoidal: $\omega_k = 1/10000^{2k/d}$, added once to the inputs. **RoPE**: rotate query/key

pairs by $m\theta_j$, $\theta_j = 10000^{-2j/d}$, so the dot product depends only on $m - n$.

Vision transformers & heads

ViT: image $\rightarrow$ patches $\rightarrow$ flatten $\rightarrow$ linear embed $\rightarrow$ prepend a learnable **CLS** token $\rightarrow$ add learnable positional embeddings $\rightarrow$ transformer *encoder* (self-attention only, no convolutions) $\rightarrow$ MLP head on the CLS token.

CNN inductive biases: **locality** (small kernels), **equivariance** (weight sharing), **invariance** (pooling) — efficient on small data. ViTs have none of these: they need more data/compute but capture global context and scale better. **Swin**: shifted windows, hierarchical, better at small scale. **ConvNeXt**: modernised CNN, comparable to ViT.

DETR: CNN backbone $\rightarrow$ encoder (+ positional encodings) $\rightarrow$ decoder with N learned **object queries** ($N = 100$ caps the detections) $\rightarrow$ shared FFN predicts class+box or “no object”; direct set prediction with a bipartite (Hungarian) matching loss; no NMS, no anchors, $\approx$ half the compute of Faster R-CNN.

Mask2Former: pixel decoder + learned queries refined by **masked cross-attention** (restricted to the previous mask’s foreground) + self-attention among queries + FFN; Hungarian matching, “no object”

padding, mask loss + classification loss. **EoMT** (CVPR 2025): strip it down to a plain ViT with **mask annealing** $\Rightarrow 4\times$ faster, -1.1 PQ.

Self-supervision: next-token prediction; masked content prediction — **MAE** masks $\sim 75\%$ of patches, encoder sees only the visible ones, a light decoder reconstructs and is then discarded.

Trap list — read this last

- Convolution = correlation **only for symmetric kernels**; “convolve” means flip first.
- Aliasing is caused by **subsampling without smoothing**, not by low-pass filtering.
- Box smoothing rings; the Gaussian does not. $FT(\text{box})=\text{sinc}$, $FT(\text{Gaussian})=\text{Gaussian}$.
- Median before Canny for salt & pepper; Gaussian alone smears the impulses.
- Harris: rotation-invariant **yes**, scale-invariant **no**, affine-invariant **no**.
- k -means: converges yes, global optimum no, NP-hard yes.
- Mean-shift finds the number of clusters itself; k -means does not.
- Homography: **4** correspondences (8 DoF); fundamental matrix: **8** (7 DoF); always solve $A\mathbf{h} = 0$ with the **smallest** singular vector.

- RANSAC k : dividing by $\log(\text{something} < 1)$ **flips the inequality**.
- $\text{rank}(F) = 2$; enforce by zeroing σ_3 ; normalise coordinates before the 8-point algorithm.
- Conv parameter counts are **independent of the input resolution**; ReLU and pooling have **zero** parameters; don’t forget the **bias** and BatchNorm’s $2C$.
- Graph cuts are globally optimal only for **binary submodular** energies; multi-label needs α -expansion.
- Integral images only give constant-time sums for **axis-aligned rectangles**.
- Weak classifiers only need $\epsilon < 0.5$; AdaBoost raises the weights of **misclassified** samples.
- A detector needs **negative** training examples, not just positives.
- Pure stacked self-attention has no non-linearity — the MLP is essential.
- When a method has an old and a modern variant (post- vs pre-norm, offline vs online mining, s - t cut vs α -expansion), **name which one you are using and why**.
- Show the full multiplication in counting questions — the arithmetic carries the marks.